

1. Basic Bernoulli Distribution Example

```
from scipy.stats import bernoulli

# Probability of success

p = 0.6

# Probability Mass Function (PMF)

print("P(X=1):", bernoulli.pmf(1, p))

print("P(X=0):", bernoulli.pmf(0, p))

# Random sampling

samples = bernoulli.rvs(p, size=10)

print("Random samples:", samples)
```

2. Visualizing the Bernoulli PMF

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.stats import bernoulli

p = 0.7

x = [0, 1]

pmf = bernoulli.pmf(x, p)

plt.stem(x, pmf, basefmt=" ", linefmt='b-', markerfmt='bo')

plt.xlabel('x (Outcome)')

plt.ylabel('P(X = x)')

plt.title(f'Bernoulli Distribution (p = {p})')

plt.ylim(0, 1)

plt.show()
```

3. Simulating Bernoulli Trials

```
import numpy as np

p = 0.4 # probability of success
n = 20 # number of trials

# Simulate 20 Bernoulli trials
trials = np.random.binomial(1, p, n)
print("Results of 20 Bernoulli trials:", trials)

# Count number of successes
successes = np.sum(trials)
print("Number of successes:", successes)
```

4. Estimating Probability from Data

```
import numpy as np

# Suppose we observed 100 Bernoulli trials
data = np.random.binomial(1, 0.65, 100)
estimated_p = np.mean(data)
print("Observed data (first 10):", data[:10])
print("Estimated p from data:", round(estimated_p, 3))
```

5. Plotting the Mean Convergence (Law of Large Numbers)

```
import numpy as np
import matplotlib.pyplot as plt

p = 0.5
n_trials = 1000

data = np.random.binomial(1, p, n_trials)
running_mean = np.cumsum(data) / np.arange(1, n_trials + 1)

plt.plot(running_mean, label='Running Mean')
plt.axhline(y=p, color='r', linestyle='--', label='True p')
```

```
plt.xlabel('Number of Trials')
plt.ylabel('Estimated Probability')
plt.title('Law of Large Numbers - Bernoulli Trials')
plt.legend()
plt.show()
```

6. Simulate a Biased Coin Toss and Estimate Bias

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import bernoulli

# True probability of Heads
true_p = 0.65

n = 1000 # number of tosses

# Generate Bernoulli samples (1 = Head, 0 = Tail)
data = bernoulli.rvs(true_p, size=n)

# Estimate probability using Maximum Likelihood Estimation (MLE)
estimated_p = np.mean(data)

print(f"Estimated p = {estimated_p:.3f}")

# Visualization of cumulative estimate
running_estimate = np.cumsum(data) / np.arange(1, n + 1)

plt.plot(running_estimate, label='Running Estimate of p')
plt.axhline(true_p, color='red', linestyle='--', label='True p')

plt.xlabel('Number of Tosses')
plt.ylabel('Estimated Probability')
plt.title('Estimating Coin Bias using Bernoulli Trials')
plt.legend()
plt.show()
```

7. Simulating a Binary Decision System

```
import numpy as np

p_true = 0.85

n = 10000

# 1 = correct detection, 0 = false alarm

sensor_output = np.random.binomial(1, p_true, n)

accuracy = np.mean(sensor_output)

false_alarm_rate = 1 - accuracy

print(f"Estimated accuracy: {accuracy:.3f}")

print(f"Estimated false alarm rate: {false_alarm_rate:.3f}")
```

8. Bernoulli Log-Likelihood Function (for parameter estimation)

```
import numpy as np

import matplotlib.pyplot as plt

# Sample Bernoulli data (e.g., 1 = success)

data = np.random.binomial(1, 0.7, 50)

p_values = np.linspace(0.01, 0.99, 100)

log_likelihood = [

    np.sum(data * np.log(p) + (1 - data) * np.log(1 - p)) for p in p_values

]

plt.plot(p_values, log_likelihood)

plt.axvline(np.mean(data), color='red', linestyle='--', label='MLE (mean)')

plt.xlabel('p')

plt.ylabel('Log-Likelihood')

plt.title('Bernoulli Log-Likelihood Function')

plt.legend()

plt.show()
```

9. Hypothesis Testing (Bernoulli Proportion Test)

```
from statsmodels.stats.proportion import proportions_ztest

# Observed data
successes = 72
n = 100
p0 = 0.8 # claimed probability

# Perform Z-test
stat, p_value = proportions_ztest(successes, n, p0)
print(f"Z-statistic: {stat:.3f}, p-value: {p_value:.4f}")

if p_value < 0.05:
    print("Reject H0: The claim is likely false.")
else:
    print("Fail to reject H0: No evidence against the claim.")
```

10. Bernoulli Process → Cumulative Success Simulation

```
import numpy as np
import matplotlib.pyplot as plt

p = 0.4
n = 200

data = np.random.binomial(1, p, n)

cumulative_successes = np.cumsum(data)
plt.plot(cumulative_successes, label='Cumulative Successes')
plt.plot(np.arange(n)*p, 'r--', label='Expected Value (np)')
plt.xlabel('Trial Number')
plt.ylabel('Cumulative Success Count')
plt.title('Bernoulli Process Over Time')
plt.legend()
plt.show()
```